SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction

InsureFlow Pro is an Agentic AI-powered orchestration system designed to solve the fragmented and manual process of insurance claims. By utilizing Z.AI's GLM-4 as the central reasoning engine, the system transforms unstructured medical reports and complex policy frameworks into automated, actionable claim verdicts.

1.1. Purpose

The system analysis document highlights the technical scope and design decisions behind the InsureFlow development. The key elements covered are:

1. **Architecture:** A native Python-based agentic platform running on a local-host environment.
2. **Data Flows:** Medical data is processed locally, ensuring high data privacy. It passes through a Vision-Language layer and is cross-referenced by the GLM API.
3. **Model Process:** The workflow includes document extraction, semantic policy matching, autonomous payout calculation, and automated communication generation.
4. **Role of Reference:** This document serves as a technical blueprint for the execution and reproduction of the InsureFlow system

1.2. Background

Currently, the insurance claim process in Malaysia is highly manual. Patients struggle with medical jargon, and auditors spend hours cross-referencing bills against 50-page policy booklet.

1. **Previous Version:** Traditional claims require manual form filling and physical document submission.
2. **Changes in Major Architectural Components:** Introduction of an "Agentic Orchestrator" that replaces human interpretation of policy clauses.
3. **New Capabilities:** Multi-modal support (handwriting recognition) and real-time financial reasoning.

© UMHackathon 2026

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
Patient	Uploads documents and receives instant claim guidance	Privacy of data, fast payout estimates, and simplified jargon.
Insurance Agent	Receives pre-verified claim packages from the AI.	Accurate data and reduced back-and-forth communication.
Development Team	Maintains and scales the Reasoning Engine.	Modular code and clear API contracts with Z.AI.

2. System Architecture & Design

2.1. High-Level Architecture

Type	Details
System	Local Web Dashboard (Streamlit)
Architecture	Native Agentic Workflow with Multi-Modal Support

Overview: InsureFlow Pro is structured as a **Native Client-Server** system. The system runs on a local Python environment to prioritize **data security and rapid prototyping**. The engine uses pypdf for extraction and Z. AI's GLM-4 as the core reasoning layer via API.2.1.2 LLM as Service Layer

The architecture integrates GLM-4 as a "Central Reasoning Engine." It receives the medical context and the policy rules locally to generate a final financial verdict without storing sensitive data on public cloud servers.

2.1.3 Dependency Diagram

- **Prompt Construction:** Constructed locally and sent to the GLM API.
- **Context Window:** Up to 2,000 tokens of medical data and policy snippets.
- **Response Handling:** The system receives JSON responses, which are parsed locally to populate the UI metrics.

© UMHackathon 2026

2.2. Technological Stack

1. **Frontend/UI: Streamlit (Python 3.12).**
2. **Processing Layer: Python Native Environment.**
3. **AI Engine: Z.AI GLM-4 (Ilmu-GLM-5.1) for live inference.**
4. **Libraries: pypdf (extraction), requests (API), pandas (data).**
5. **Environment: Local Host Deployment (for maximum data privacy).**

3. Functional Requirements & Scope

3.1 Minimum Viable Product (MVP)

#	Features	Description
1	Multi-Modal Data Extraction	Extracting data from digital PDFs and photos of handwritten reports.
2	Agentic Policy Matching	Semantically matching a diagnosis to a specific policy benefit.
3	Autonomous Calculation	Automatically calculating deductibles and co-insurance.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Reliability	100% Uptime for demo.	Implementation of a Local Heuristic Engine that triggers if the API times out
Data Privacy	Localized processing	No sensitive medical data is stored on external cloud hosting
Usability	Cinematic user experience.	"Any Key" enter, CSS animations, and keyboard shortcuts (Ctrl + Enter).

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

- Heuristic Fallback: If Z.AI returns a 504/502 error, the system switches to internal pattern-matching to maintain the user flow.
- Match Accuracy: Evaluated by comparing AI verdicts against known insurance policy results (Ahmad, Siti, and Chong test cases).

4.2. Monitoring:

4.2.1. InsureFlow Pro utilizes a multi-layered monitoring plan to ensure the reliability of the Agentic Workflow:

- **API Health Monitoring:** The system continuously tracks the response status of the Ilmu.ai endpoint. If a 504 (Gateway Timeout) or 404 (Not Found) error is detected, the system triggers an automatic internal alert and switches to the **Local Heuristic Engine** to prevent a total system blackout.
- **Latency Tracking:** Since GLM reasoning can take up to 50–90 seconds for complex policies, the system monitors "Reasoning Duration." If the duration exceeds the 90-second threshold, the system gracefully degrades to a faster, pattern-based local scan to maintain user experience.
- **Accuracy Audit:** The system logs cases where the AI Match Score is below 70%. These cases are flagged for "Manual Human Review," ensuring that the AI does not make high-stakes financial decisions without sufficient confidence.
- **System Health Dashboard:** Since the app runs on a Streamlit framework, the "System Active" indicator in the sidebar acts as a real-time heartbeat monitor for the user, confirming that the back-end orchestration layer is functional.

4.3. Assumptions:

State key operational & environmental conditions that you assumed to be valid while development of system and deployment.

- **a.** Users have a stable internet connection to communicate with the Ilmu.ai API for live reasoning.
- **b.** medical reports provided (PDF or JPG) are legible enough for the Vision-Language model to perform character recognition.
- **c.** The insurance policy documents provided follow a standard clause-based structure (e.g., specific sections for benefits and exclusions).

© UMHackathon 2026

4.4. External Dependencies

Tools	Purpose	Risks
Ilmu.ai GLM API	Core agentic reasoning, handwriting interpretation, and multi-modal data extraction.	**High** - Potential server timeout during peak traffic (Mitigated by Local Heuristic Fallback).
Python 3.12	Core programming language for logic and back-end orchestration.	**Low** - Requires environment setup on the reviewer's side.
Streamlit	UI Framework for the cinematic dashboard and file management.	**Low** - Standard framework for AI prototypes.
PyPDF2 / pypdf	Digital extraction of unstructured text from insurance policy booklets.	**Low** - Library is stable and localized.

5. Project management & team Contributions

5.1. Project Timeline

- Day 1: Problem identification, UI/UX prototyping in Streamlit.
- Day 2 (Morning): Logic orchestration, PDF extraction setup.
- Day 2 (Night): API integration, cinematic animations, and multi-modal testing.
- Day 3: Deployment and Final Documentation.

5.2. Team Members Role (Team Midas UTeM)

- Member A (Developer/Architect): Full-stack Python development, Z.AI API integration, and Heuristic Logic design.
- Member B (Product/Pitcher): PRD/SAD Documentation, UI/UX Styling (CSS), and Cinematic Storyboarding.

© UMHackathon 2026